

PARABANK AUTOMATION PROJECT REPORT

Selenium WebDriver • Java • TestNG • Maven

[GitHub Link](#)

Wipro Capstone Project

Project Name	ParaBankAutomationFramework
Prepared By	Suraj Shaw
Application Under Test	ParaBank Online Banking
Framework Type	Hybrid Page Object Model and Data-Driven Framework
Execution Result	8 Passed / 8 Executed (100%)

Test execution completed successfully with no failures or skipped tests.

1. Project Description

The ParaBank Automation Framework is a complete web-application testing project developed to automate the major banking workflows of the ParaBank demo application. The framework validates end-to-end user journeys such as registration, valid and invalid login, opening a new account, transferring funds, checking the remaining balance, and paying bills.

The solution uses Java and Selenium WebDriver for browser automation, TestNG for execution and assertions, Maven for dependency and build management, Page Object Model for maintainable page classes, Excel for test-data storage, DataProvider for multiple datasets, listeners for screenshots and reporting, Jenkins for CI execution, and Docker support for portable builds.

2. Project Objectives

- Automate the most important ParaBank banking workflows.
- Design a reusable and maintainable test framework using Page Object Model.
- Support data-driven testing through Excel and TestNG DataProvider.
- Validate both positive and negative scenarios.
- Generate execution evidence through screenshots and test reports.
- Support continuous integration using Jenkins and containerized execution using Docker.

3. Scope of Testing

Module	Coverage	Type
Registration	Create a new user and save generated credentials	Positive
Login	Login with valid credentials read from Excel	Positive
Invalid Login	Validate invalid username and password	Negative
Empty Login	Validate empty username and password	Negative
Open Account	Open a new savings account	Positive
Transfer Funds	Transfer amount between different accounts	Positive
Balance Validation	Verify updated remaining balance	Validation
Bill Payment	Execute bill payment with multiple datasets	Data-driven

4. Technology Stack

Technology / Tool	Purpose
Java	Programming language used to develop the framework
Selenium WebDriver	Web browser automation
TestNG	Test execution, assertions, priorities, listeners, and DataProvider
Maven	Dependency management and build execution
Page Object Model	Reusable and maintainable page classes
Apache POI / Excel	External test-data handling
Extent Reports	Readable HTML execution reporting
Jenkins	Continuous integration and scheduled execution
Docker	Portable and consistent execution environment
GitHub	Source-code version control and collaboration

JIRA	Defect tracking and bug reporting
------	-----------------------------------

5. Framework Architecture

The framework follows a hybrid design combining Page Object Model and data-driven testing. The major layers are described below:

Base Layer: Initializes the WebDriver, opens the application URL, and closes the browser after execution.

Page Layer: Stores locators and reusable actions for Login, Registration, Open Account, Transfer Funds, Bill Pay, and Account Overview pages.

Test Layer: Contains TestNG test classes, assertions, priorities, and test flow validation.

Data Layer: Reads registration and login data from Excel and supplies multiple bill-payment datasets.

Utility Layer: Provides Excel handling, configuration reading, screenshots, waits, and other reusable functions.

Listener and Reporting Layer: Captures failure screenshots and writes execution details to reports.

CI/CD Layer: Runs Maven test commands through Jenkins and supports Docker-based execution.

6. Test Plan

Test Plan Item	Details
Objective	Validate the core functional and negative scenarios of the ParaBank application.
Test Types	Functional, smoke, regression, negative, UI, and data-driven testing.
Browser	Google Chrome
Entry Criteria	Application URL available, framework configured, test data available, and browser installed.
Exit Criteria	All planned tests executed, defects recorded in JIRA, and execution report generated.
Deliverables	Source code, test cases, TestNG results, Extent Report, screenshots, Jenkinsfile, Dockerfile, JIRA bugs, and analysis report.
Risks	Application downtime, test-data dependency, browser updates, and synchronization delays.

7. Test Execution Summary

Total	Passed	Failed	Skipped	Pass Rate
-------	--------	--------	---------	-----------

8	8	0	0	100%
---	---	---	---	------

Pass Percentage = 100%

8. Detailed Test Case Results

Test Case	Description	Execution	Status
verifyRegistration	Register a new user and save credentials to Excel	1	Passed
verifyValidLogin	Login using valid Excel credentials	1	Passed
verifyOpenAccount	Open a new banking account	1	Passed
verifyTransferFunds	Transfer funds and validate remaining balance	1	Passed
verifyBillPayment	Bill payment using DataProvider dataset 1	1	Passed
verifyBillPayment	Bill payment using DataProvider dataset 2	1	Passed
invalidLoginTest	Validate incorrect login credentials	1	Passed
emptyLoginTest	Validate empty login credentials	1	Passed

9. Execution Observations

- A unique user was registered successfully and the generated username was stored for later tests.
- Valid login succeeded using credentials read from the LoginData Excel sheet.
- A new account was created successfully.
- The fund transfer was completed between two different account numbers.
- The source balance was reduced from 415.5 to 405.5 after transferring 10.0.
- The remaining balance validation passed.
- Bill Payment executed successfully for two separate datasets.
- Invalid-login and empty-login validation tests passed.
- No test failed and no test was skipped.

10. JIRA Defect Report Summary

The following application issues were identified during negative and validation testing and can be recorded in JIRA as Bug issues:

ID	Bug Summary	Expected Result	Observed Result	Priority
----	-------------	-----------------	-----------------	----------

BUG-01	Bill Pay accepts zero amount	Zero-value payment should be rejected	Payment is accepted	High
BUG-02	Bill Pay accepts negative amount	Negative amount should be rejected	Payment is accepted	High
BUG-03	Transfer allows same source and destination account	Accounts should be different	Transfer is accepted	Medium
BUG-04	Transfer Funds accepts zero amount	Zero-value transfer should be rejected	Transfer is accepted	Medium

11. Sample JIRA Bug Format

Issue Type	Bug
Summary	Bill Pay allows payment with zero amount
Description	The ParaBank Bill Pay module accepts an amount of 0 and completes the transaction.
Steps to Reproduce	1. Login to ParaBank. 2. Open Bill Pay. 3. Enter valid payee data. 4. Enter amount 0. 5. Click Send Payment.
Expected Result	The application should display a validation message and prevent the transaction.
Actual Result	The bill payment is completed successfully.
Priority	High
Evidence	Attach screenshot and automation execution output.

12. Conclusion

The ParaBank Automation Framework successfully automated the planned banking workflows and achieved a **100% pass rate** for the latest TestNG execution. The project demonstrates practical implementation of Selenium WebDriver, Java, TestNG, Maven, Page Object Model, Excel-based data handling, reusable utilities, reporting, Jenkins, Docker, GitHub, and JIRA defect management.

The framework is suitable for smoke and regression execution and can be extended with cross-browser testing, parallel execution, additional banking modules, and environment-based configuration.

Prepared By	Status
Suraj Shaw	Completed

[illegible]

